Listing 9.4. Phone book with data loaded from event stream

```
fs = require 'fs'
{EventEmitter} = require 'events'

withEvents = (emitter, event) ->
  pipeline = []
  data = []

  reset = ->                          # Reset the event "log" when started.
    pipeline = []

  run = ->
    result = data
    for processor in pipeline
      if processor.filter?
        result = result.filter processor.filter
      else if processor.map?
        result = result.map processor.map
    result                            # Process all the filters and maps against the event log.

  emitter.on event, (datum) ->        # Add event listeners to the emitter.
    data.push datum


  filter: (filter) ->
    pipeline.push {filter: filter}
    @
  map: (map) ->
    pipeline.push {map: map}
    @                                 # Create the fluent interface by returning @ in the methods.
  evaluate: ->
    result = run()
    reset()
    result

class CSVRowEmitter extends EventEmitter      # Extend EventEmitter to emit rows from a file in CSV format.

  valid = (row) ->
      /[^,]+,[^,]+,[^,]+/.test row

  constructor: (source) ->
    @remainder = ''
    @numbers = []
    stream = fs.createReadStream source, {flags: 'r', encoding: 'utf-8'}
    stream.on 'data', (data) =>
      chunk = data.split /\n/
      firstRow = chunk[0]
      lastRow = chunk[chunk.length-1]
      if not valid firstRow and @remainder
        chunk[0] = @remainder + firstRow
      if not valid lastRow
        @remainder = lastRow
        chunk.pop()
      else @remainder = ''

      @emit('row', row) for row in chunk when valid row
```